

Inteligencia Artificial

Clase 3: Algoritmos de búsqueda



En la clase anterior...

- Agentes racionales
- Tipos de agentes
- Problemas de búsqueda



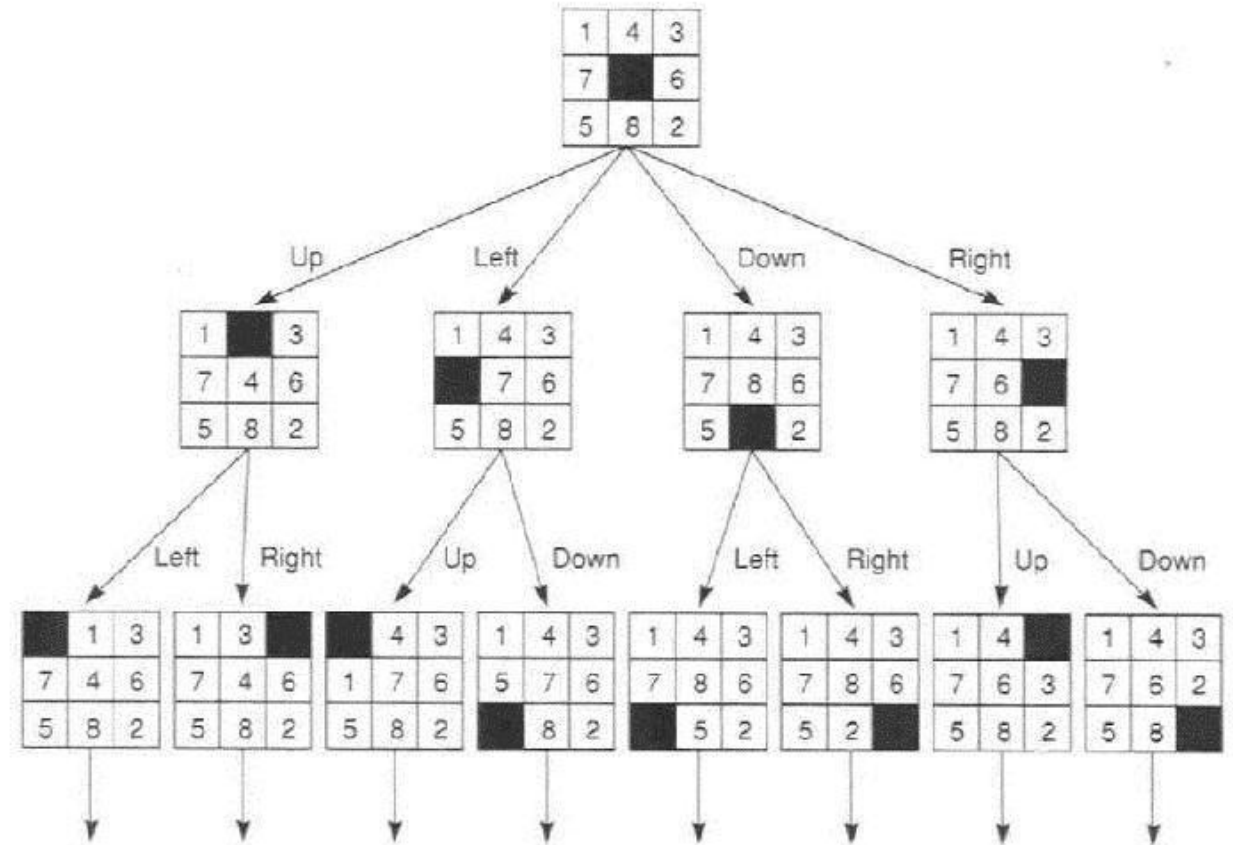
Para el día de hoy

- Problemas de búsqueda
- Búsqueda en...
 - profundidad
 - anchura
 - profundidad iterativa
 - Costo uniforme



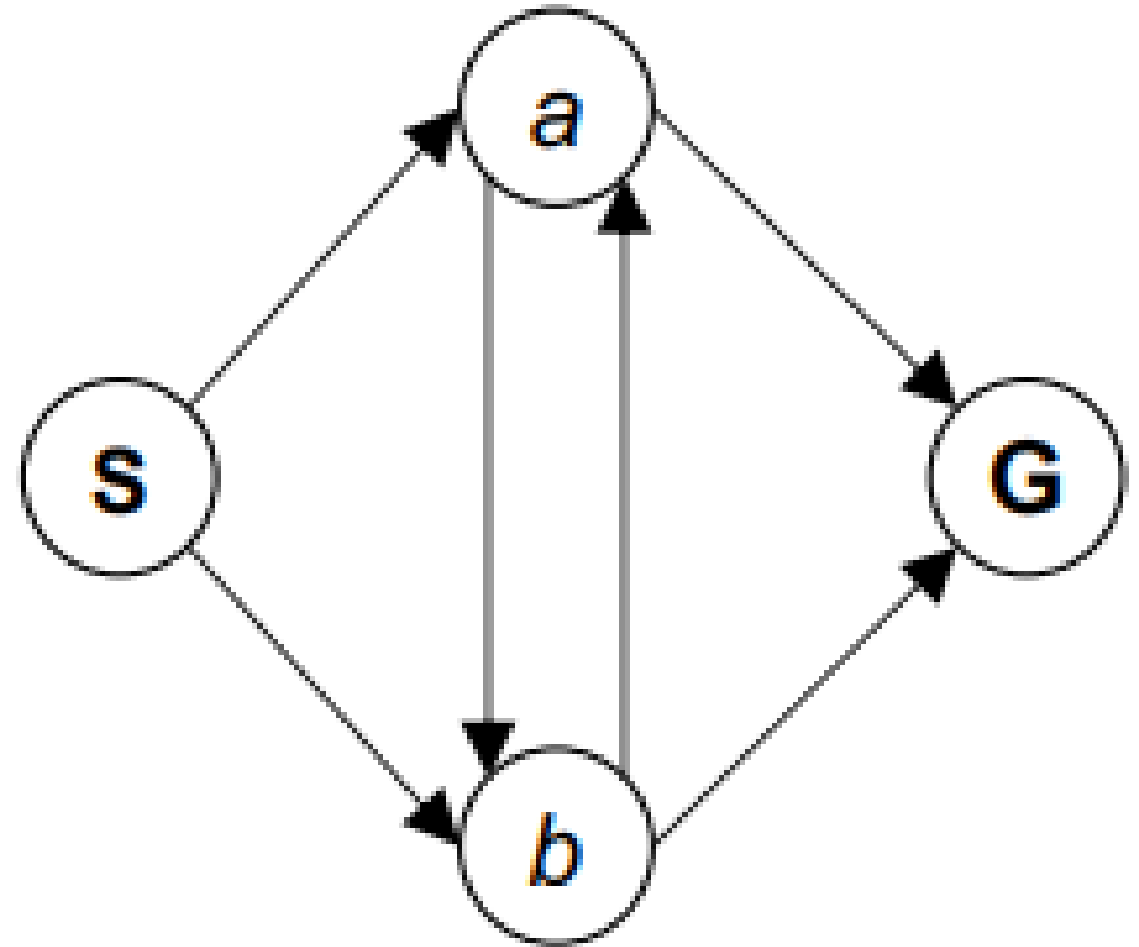
Búsqueda en arboles

- El estado inicial es la raíz
- Los hijos corresponden a los sucesores
- Los nodos muestran los estados pero corresponden a planes que alcanzan esos estados
- Rara vez se puede construir todo el árbol
- Propiedades
 - b es el factor de ramificación
 - m es la máxima profundidad
 - Soluciones en diferentes lugares



Grafos de espacio de estados vs árboles de búsqueda

- ¿Qué tan grande es el árbol de búsqueda?



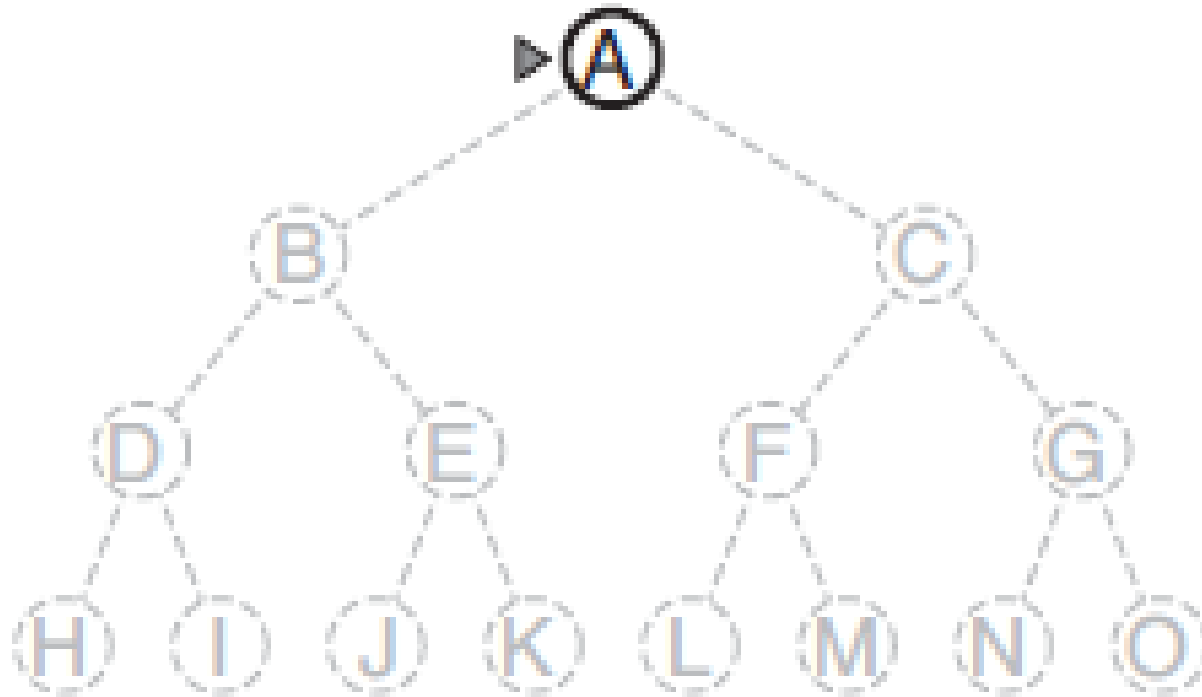
```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

Algoritmo general para buscar en un árbol

- Ideas importantes
 - Fringe (la frontera)
 - Expansión
 - Estrategia de exploración

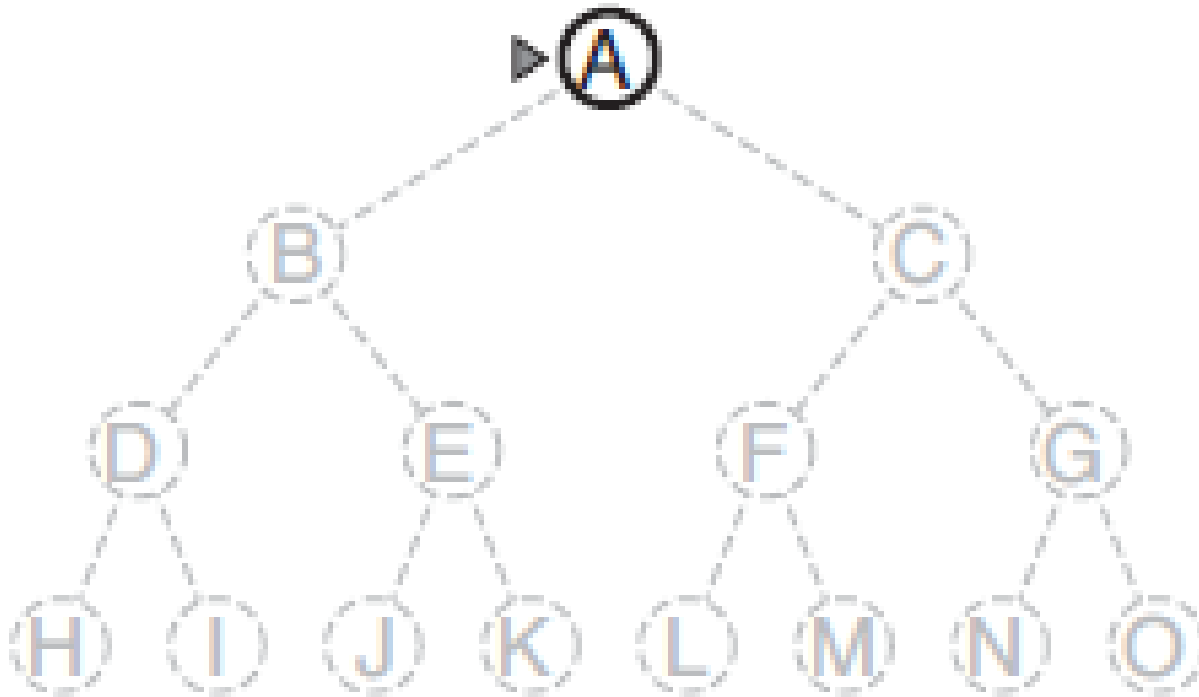
¿Cuál nodo de la
frontera explorar?

Búsqueda en profundidad (DFS)



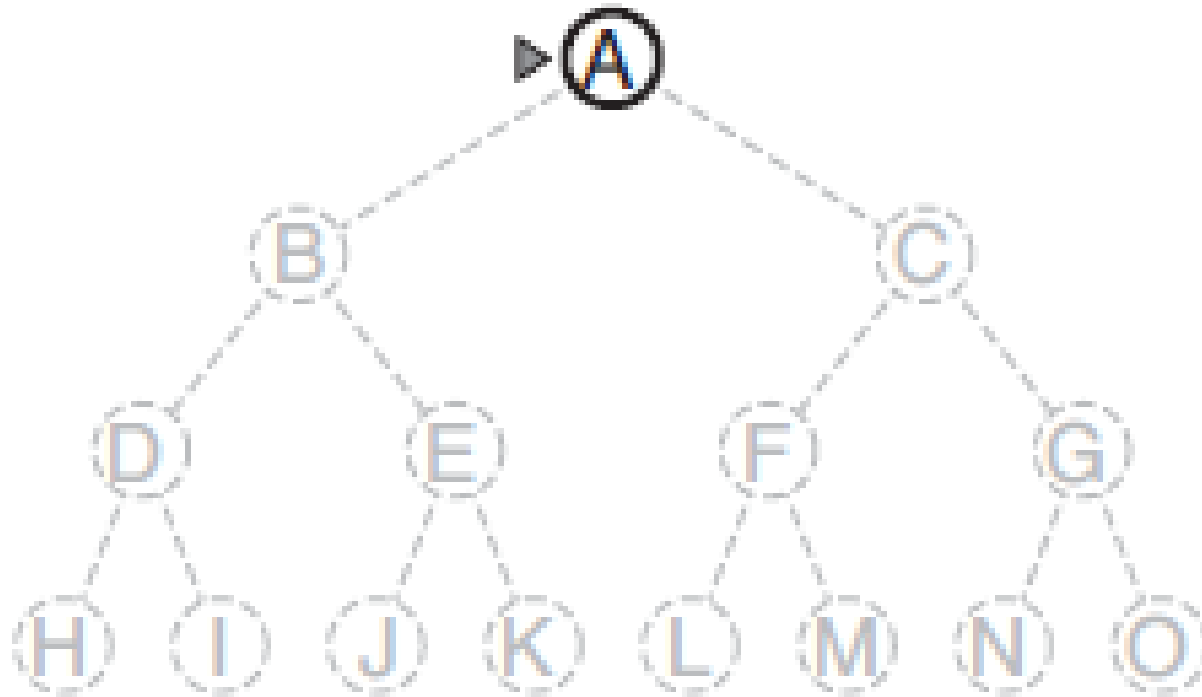
- Se extiende primero el nodo más profundo

Propiedades de búsqueda en profundidad



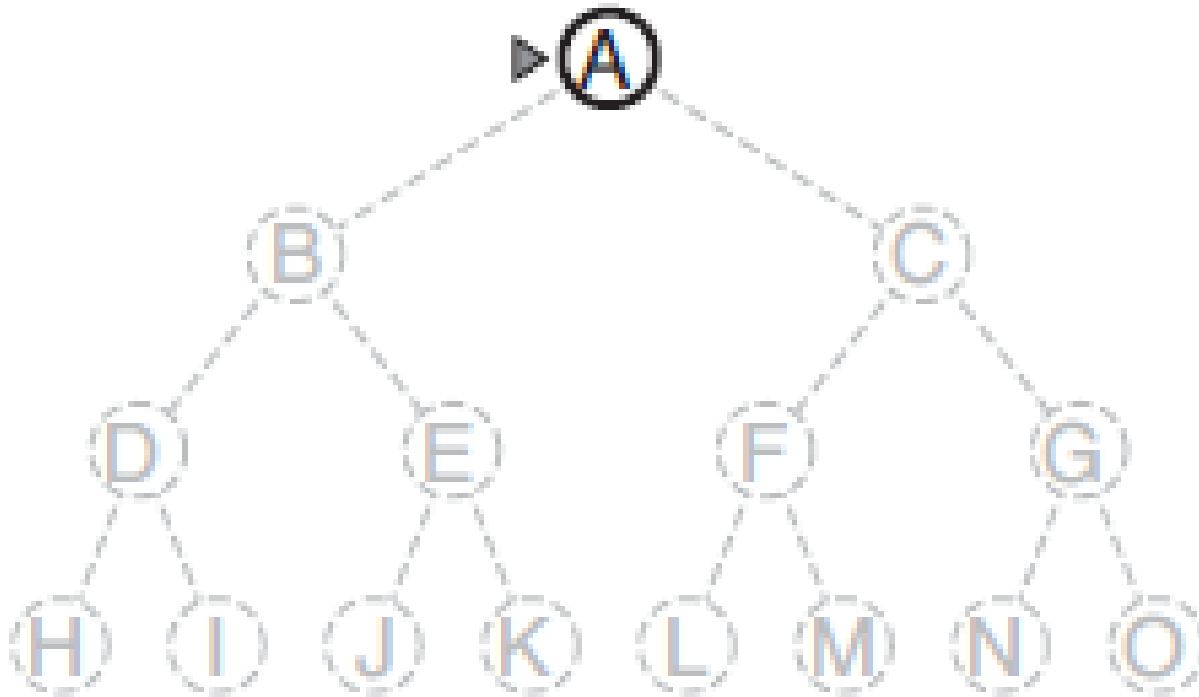
- Completo: ¿garantiza que encuentra una solución?
- Óptimo: ¿garantiza que encuentra el camino con menor costo?
- ¿Complejidad en tiempo?
- ¿Complejidad en espacio?

Búsqueda en anchura (BFS)

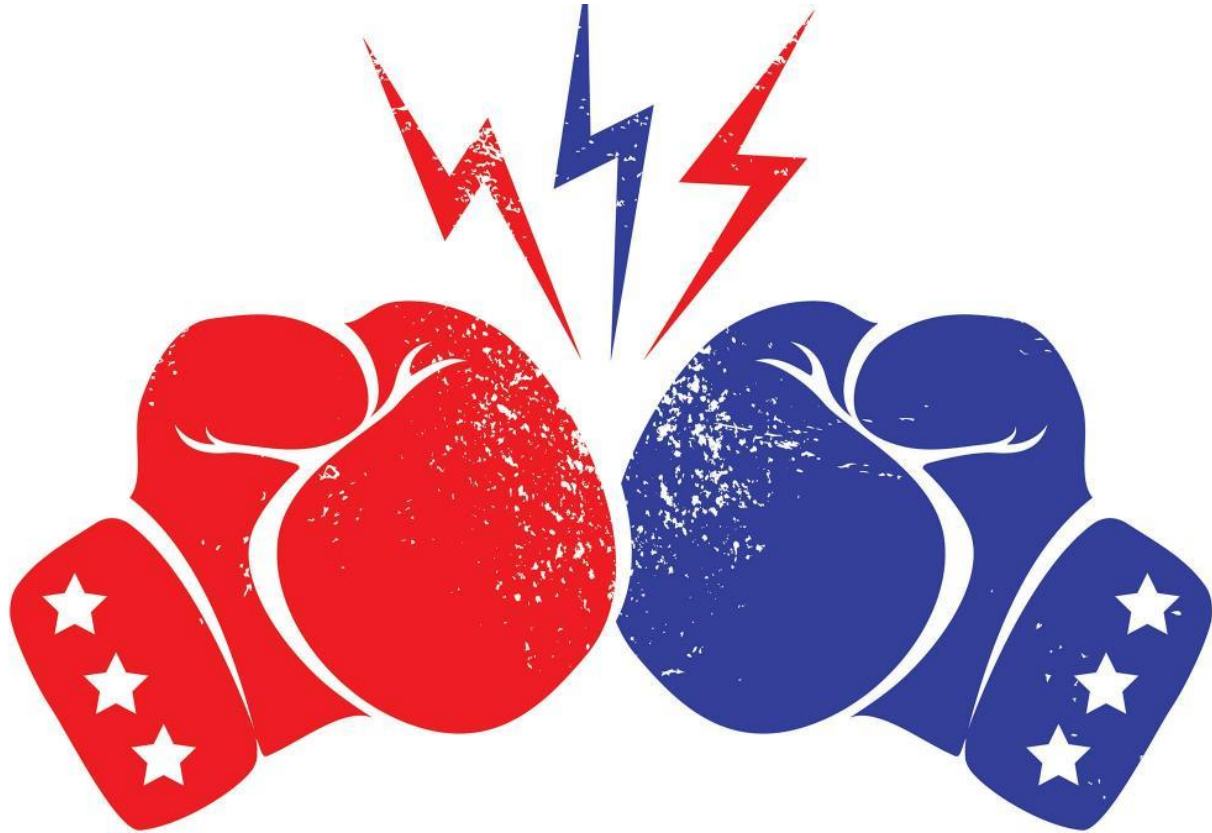


- Se extiende primero el nodo más superficial

Propiedades de búsqueda en anchura



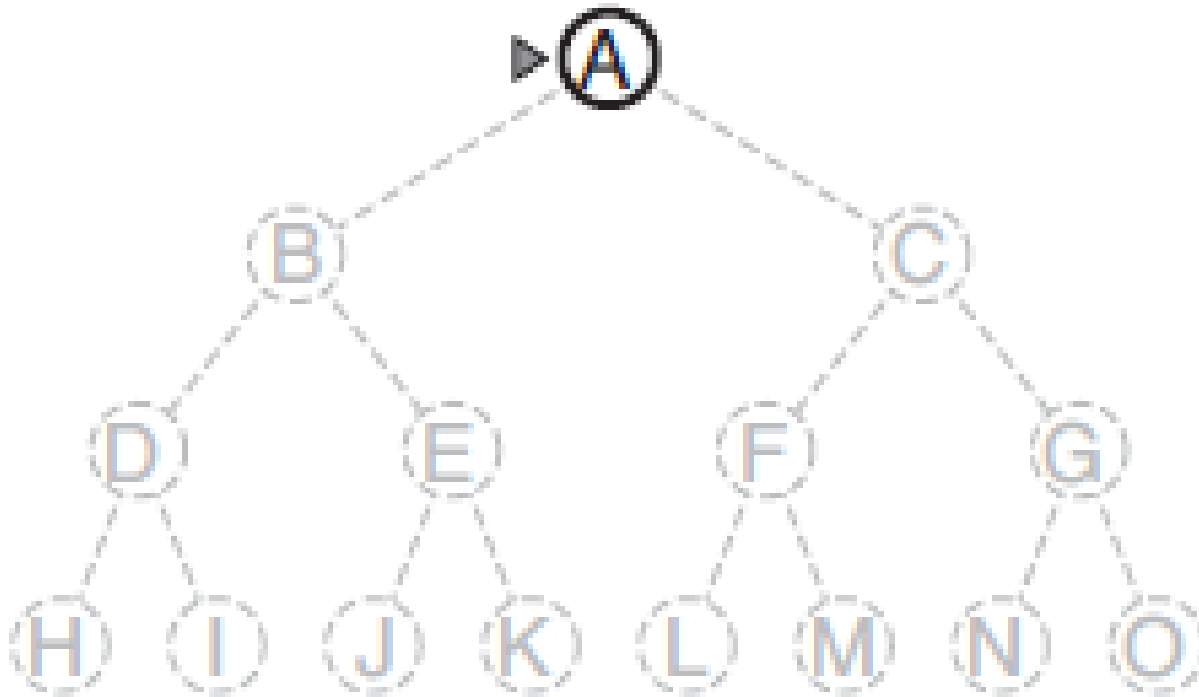
- Completo: ¿garantiza que encuentra una solución?
- Optimo: ¿garantiza que encuentra el camino con menor costo?
- ¿Complejidad en tiempo?
- ¿Complejidad en espacio?



DFS vs BFS

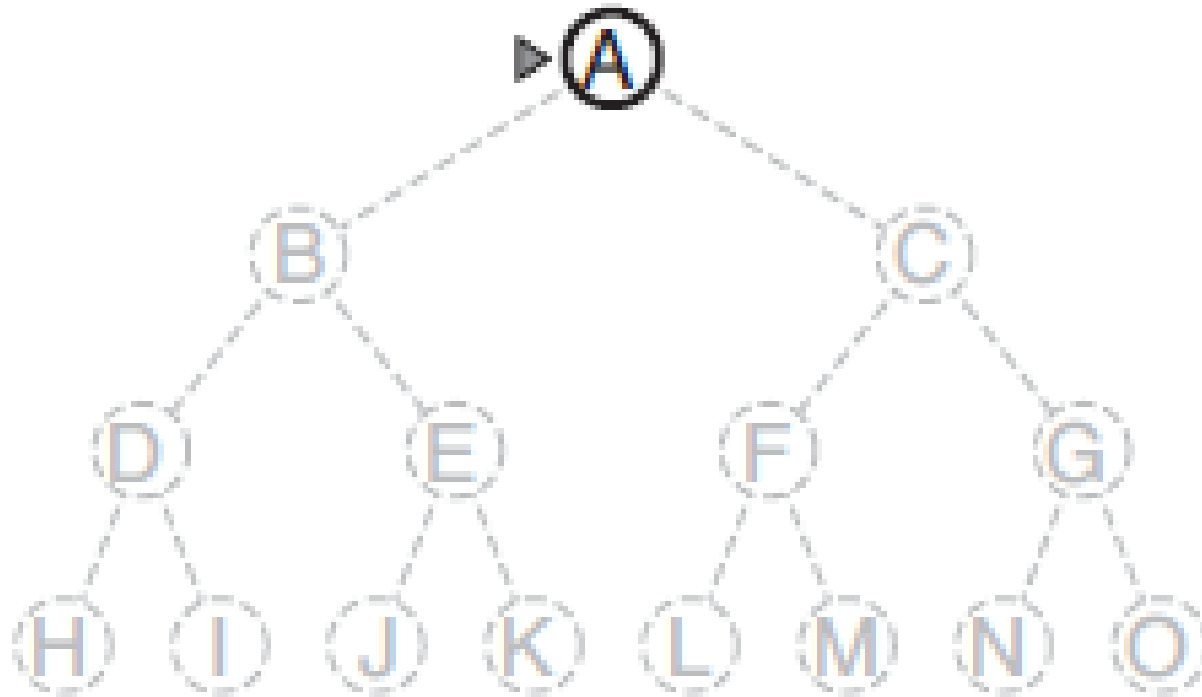
¿Cuándo sería mejor utilizar cada algoritmos?

Búsqueda en profundidad iterativa



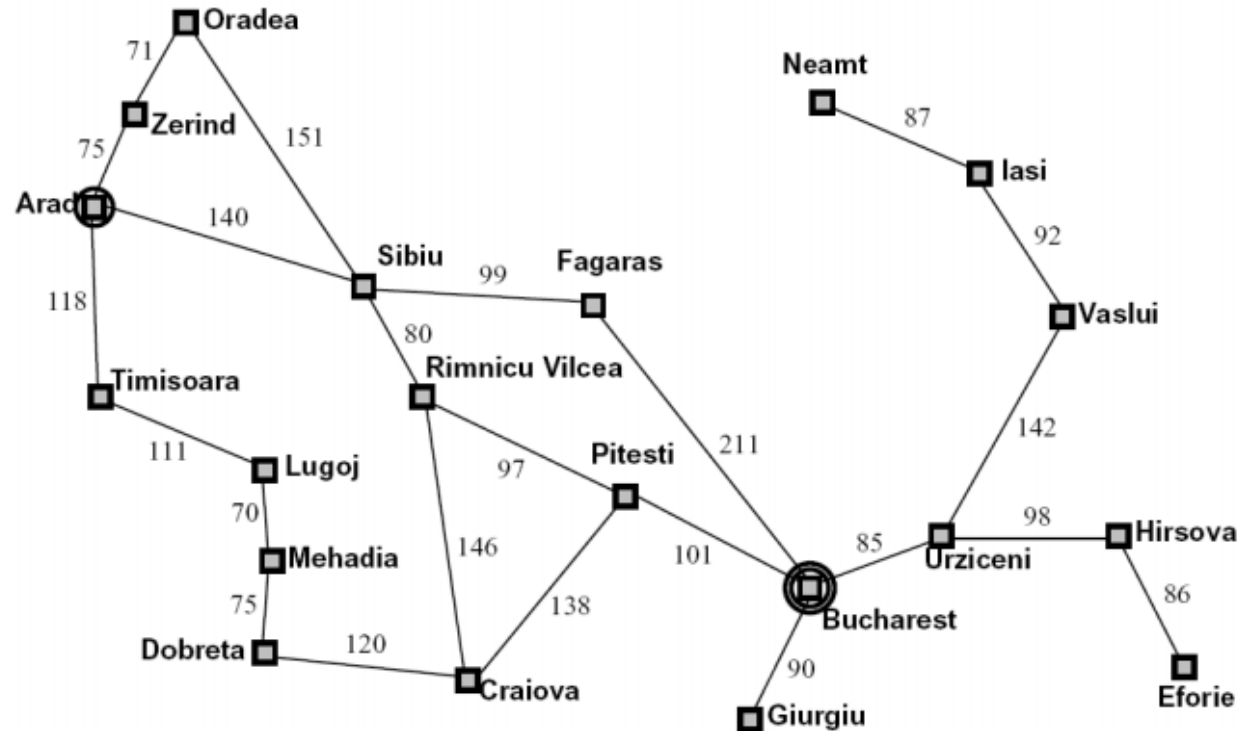
- Idea: usar las ventajas de DFS y BFS
 - Ejecutar DFS con limite 1 en profundidad
 - Ejecutar DFS con limite 2 en profundidad
 - Ejecutar DFS con limite 3 en profundidad...

Propiedades de búsqueda en profundidad iterativa



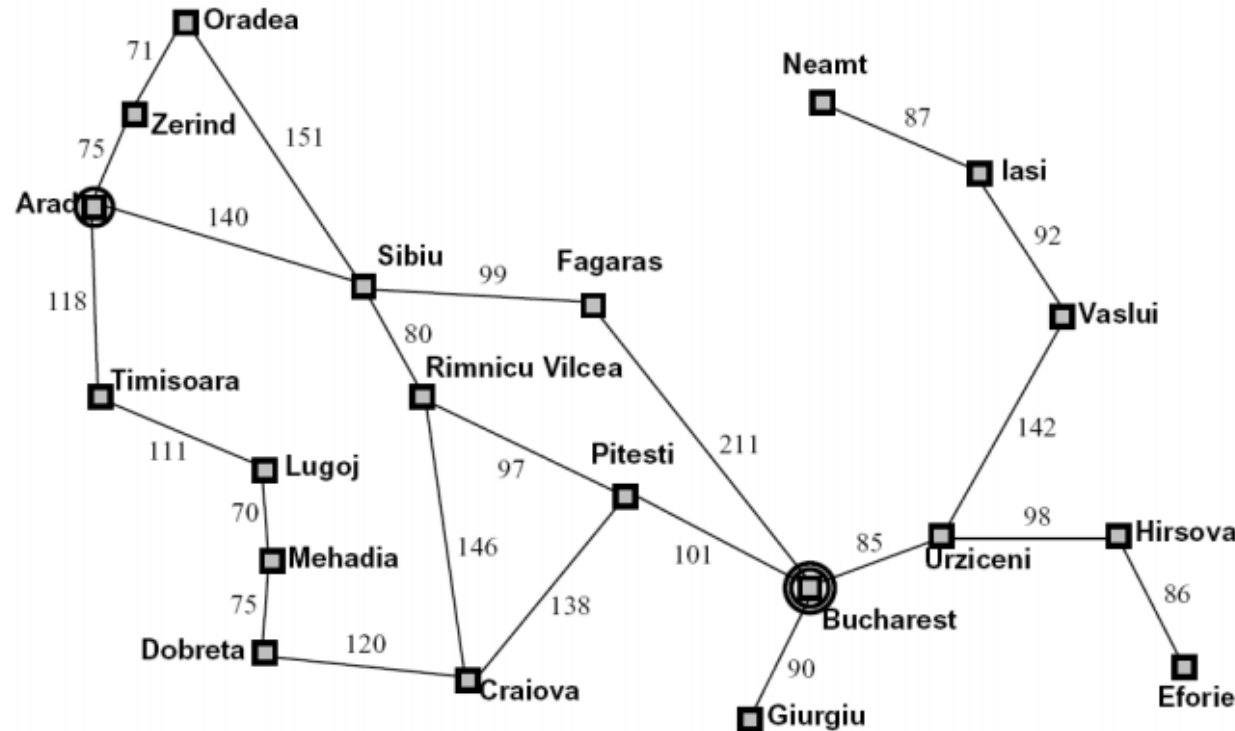
- Completo: ¿garantiza que encuentra una solución?
- Optimo: ¿garantiza que encuentra el camino con menor costo?
- ¿Complejidad en tiempo?
- ¿Complejidad en espacio?

Búsqueda de costo uniforme



- Expandir primero el nodo con el menor costo

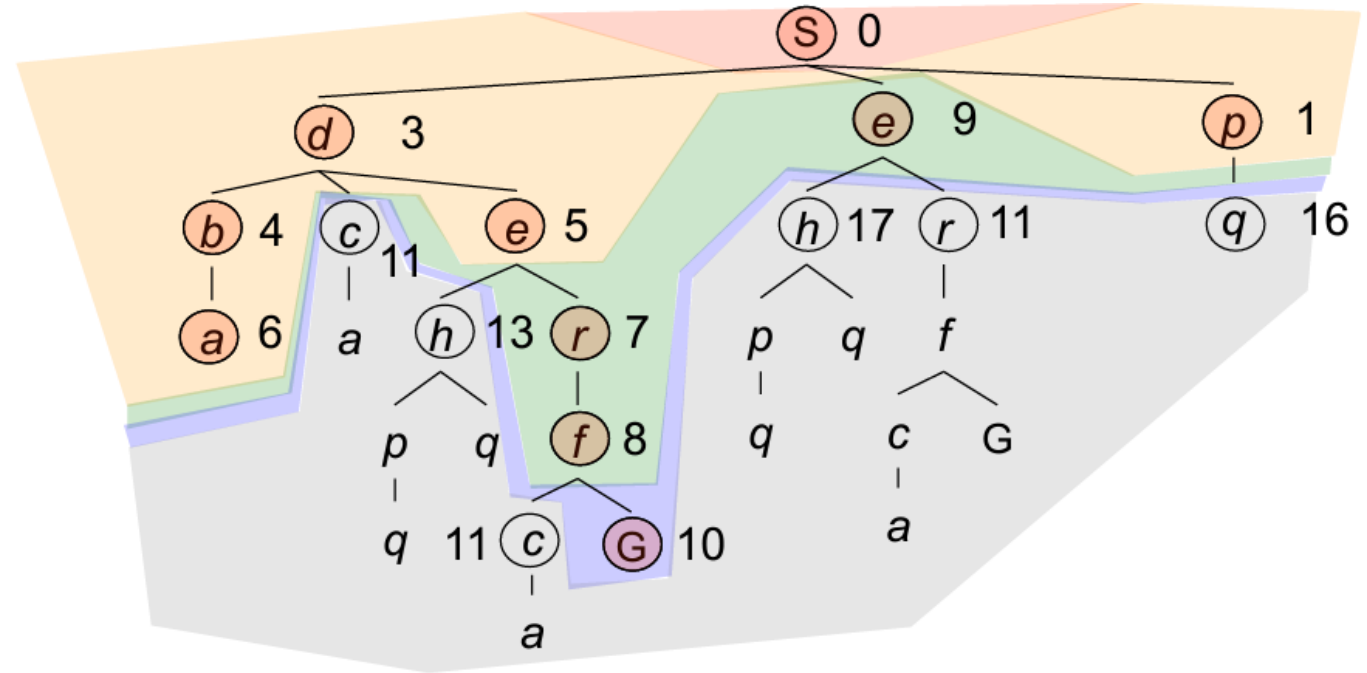
Propiedades de búsqueda de costo uniforme



- Completo: ¿garantiza que encuentra una solución?
- Optimo: ¿garantiza que encuentra el camino con menor costo?
- ¿Complejidad en tiempo?
- ¿Complejidad en espacio?

Búsqueda de costo uniforme

- Procesa los nodos que sean más baratos que la solución óptima
- Sea C^* el costo de la solución óptima
- Sea el costo del arco al menos ϵ
- La profundidad "efectiva" sería $\frac{C^*}{\epsilon}$
- Complejidad
 - Tiempo: $O(b^{\frac{C^*}{\epsilon}})$
 - Espacio: $O(b^{\frac{C^*}{\epsilon}})$

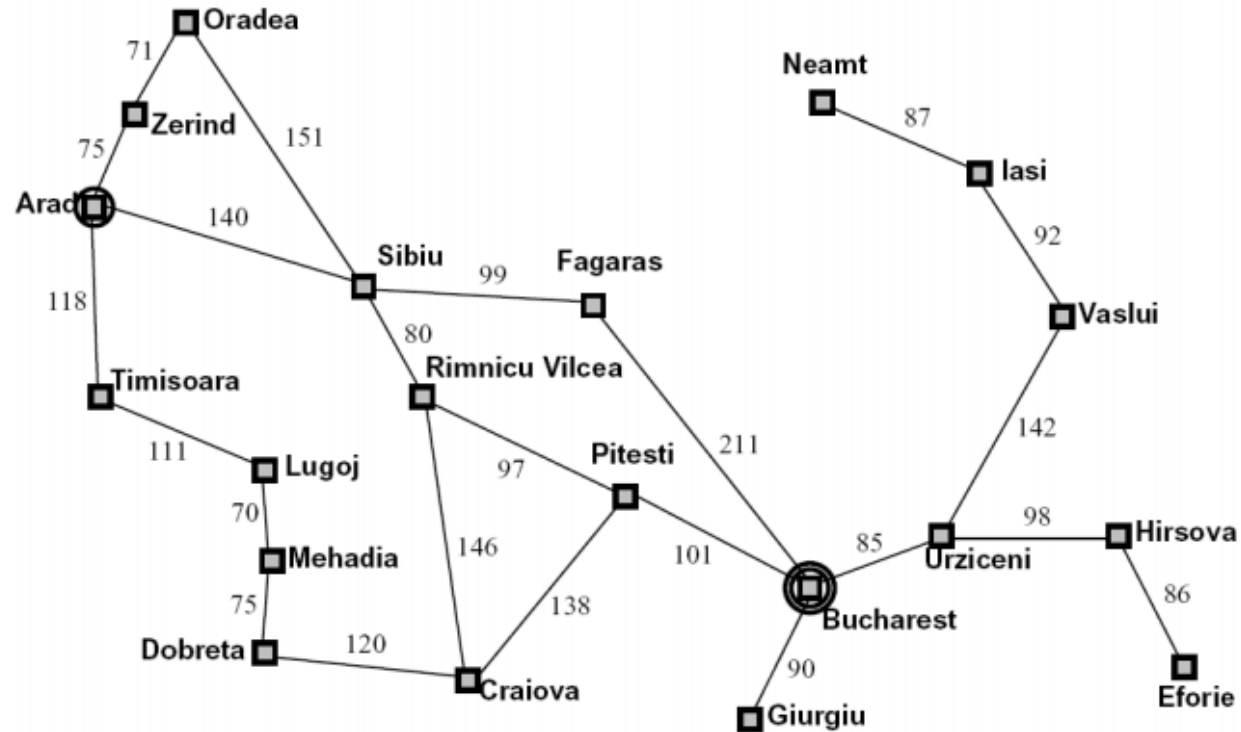




La estrategia única

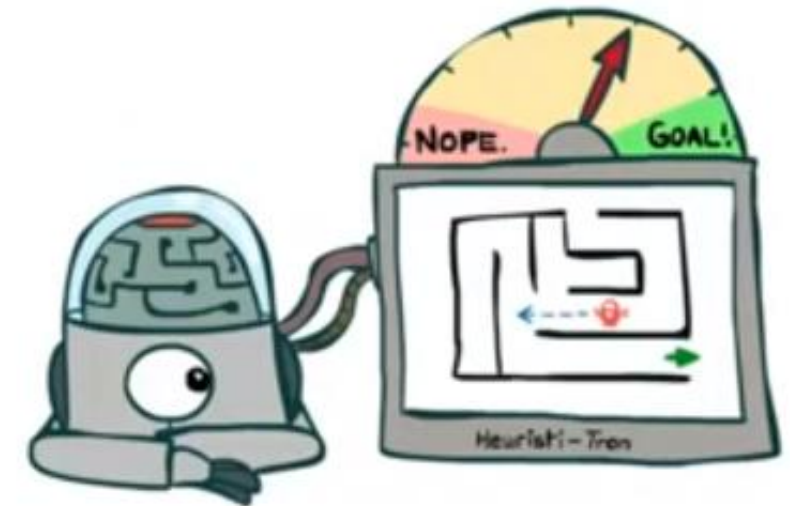
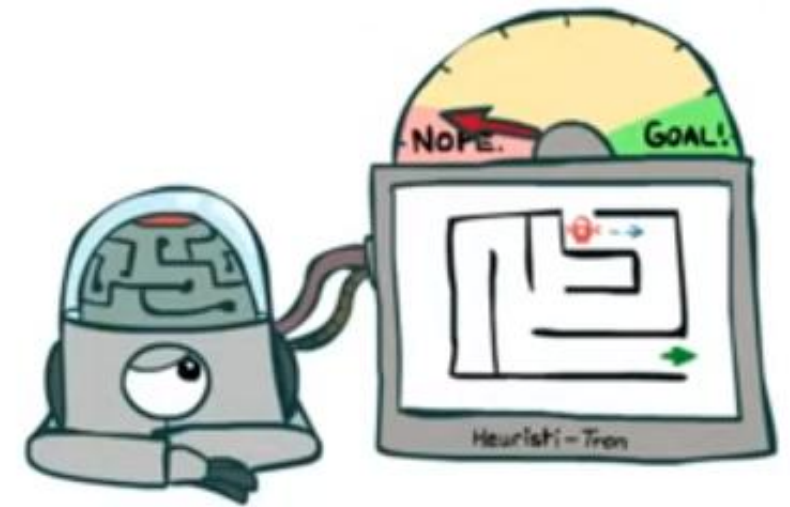
- Todos los algoritmos pueden ser implementados solo cambiando la estrategia en la frontera
- Recomendación: ¡usar una cola de prioridad!

Nuestra tarea

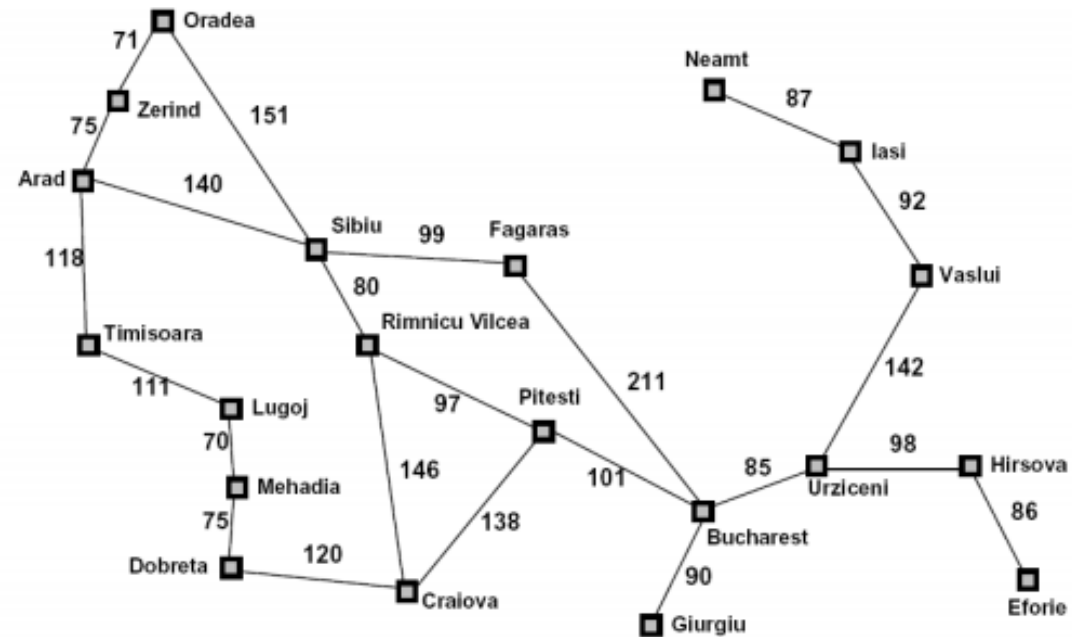


Búsqueda informada y heurísticas

- ¡No hemos utilizado información del espacio de búsqueda!
- Una heurística es:
 - Una función que estima que tan cerca estamos de la meta
 - Se diseña por problema
 - Ejemplo: distancia entre dos ciudades



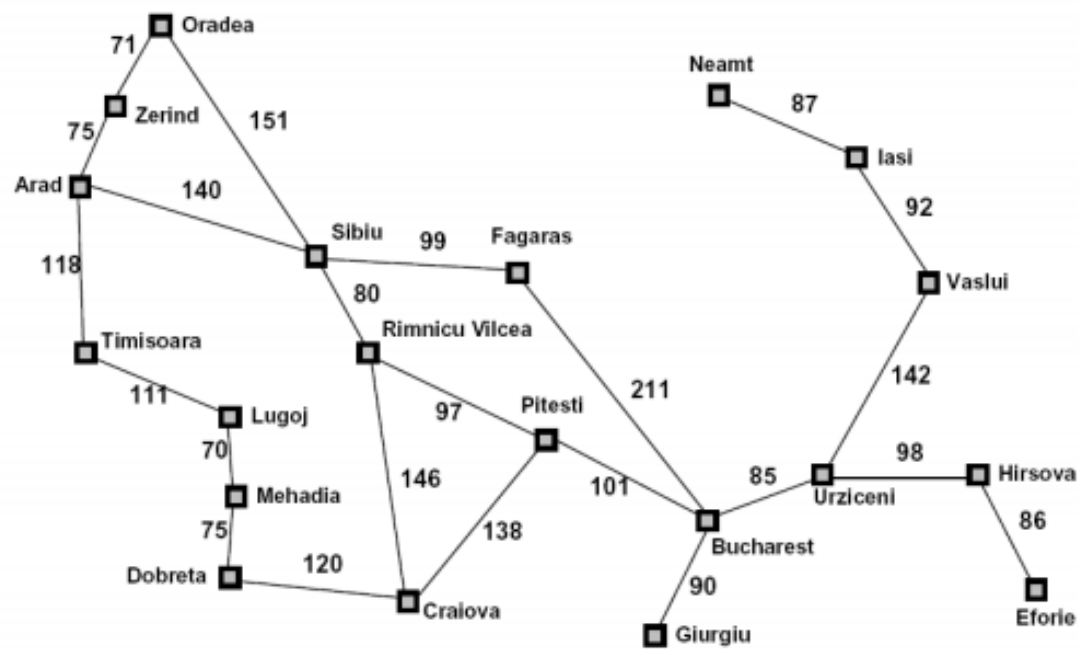
Ejemplo en el problema de ciudades



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Búsqueda voraz

- Expandir el nodo que se ve más cerca
- ¿Qué puede salir mal?
- Peor escenario: búsqueda en profundidad mal guiado



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

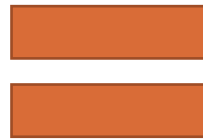
¿Qué podemos hacer?



UCS



Greedy

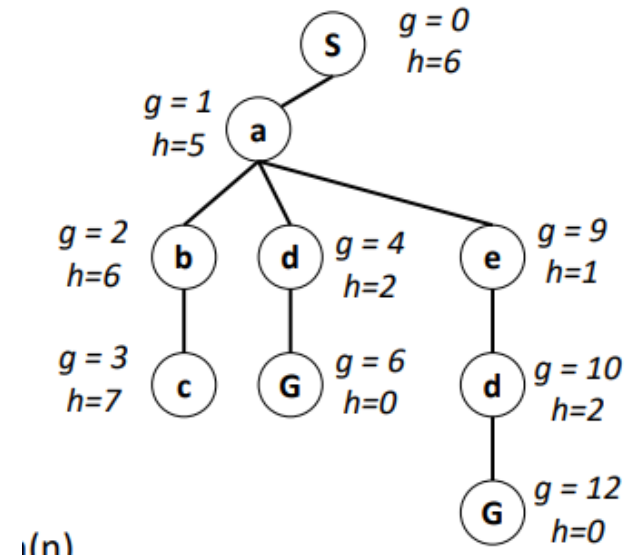
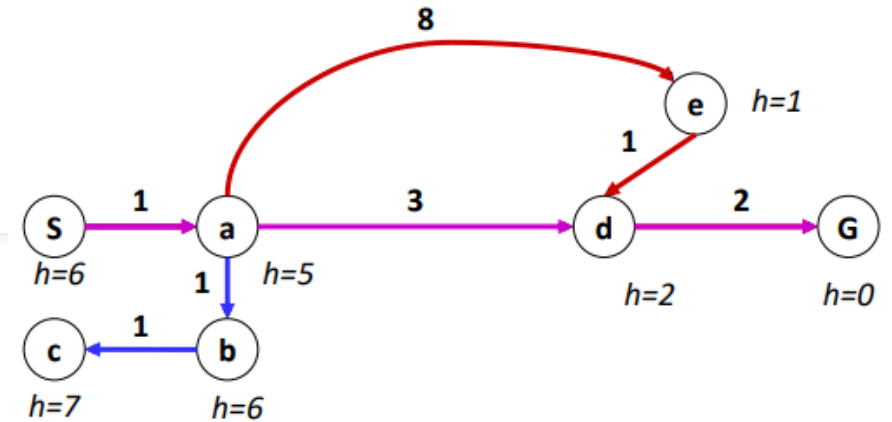


A*

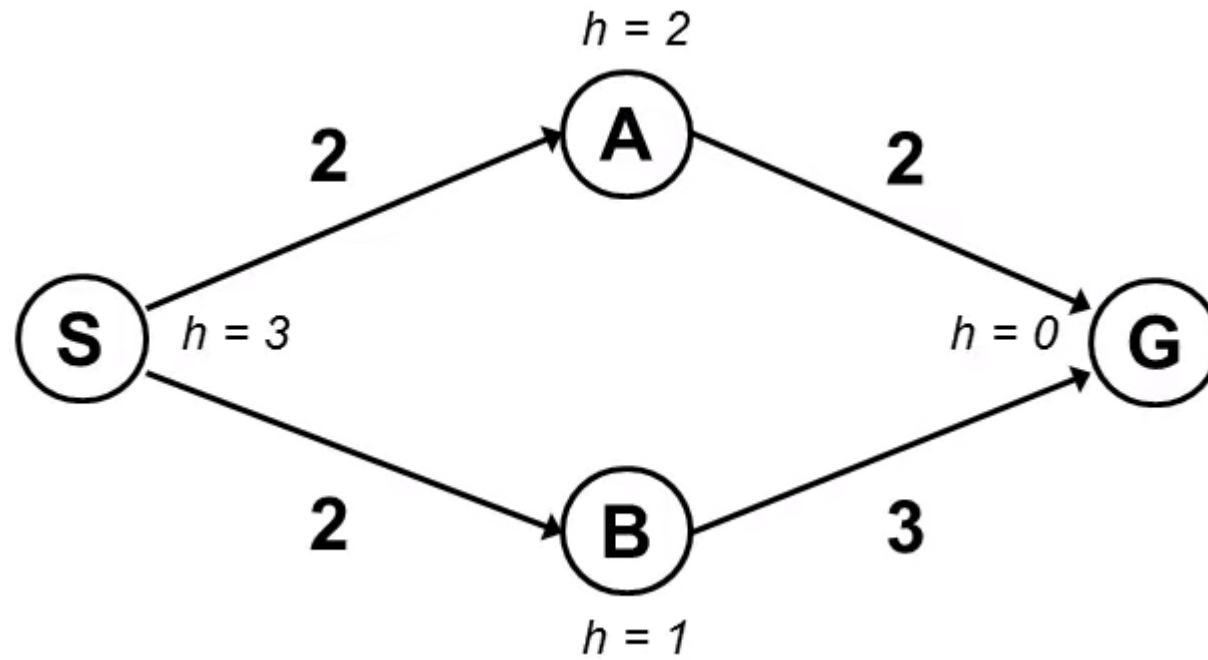
$A^* = \text{Costo uniforme} + \text{voraz}$

- Ordena la búsqueda de acuerdo con la suma de

$$f(n) = g(n) + h(n)$$
- Donde:
 - $g(n)$ es el costo utilizado por costo uniforme
 - $h(n)$ es el costo utilizado por la búsqueda voraz



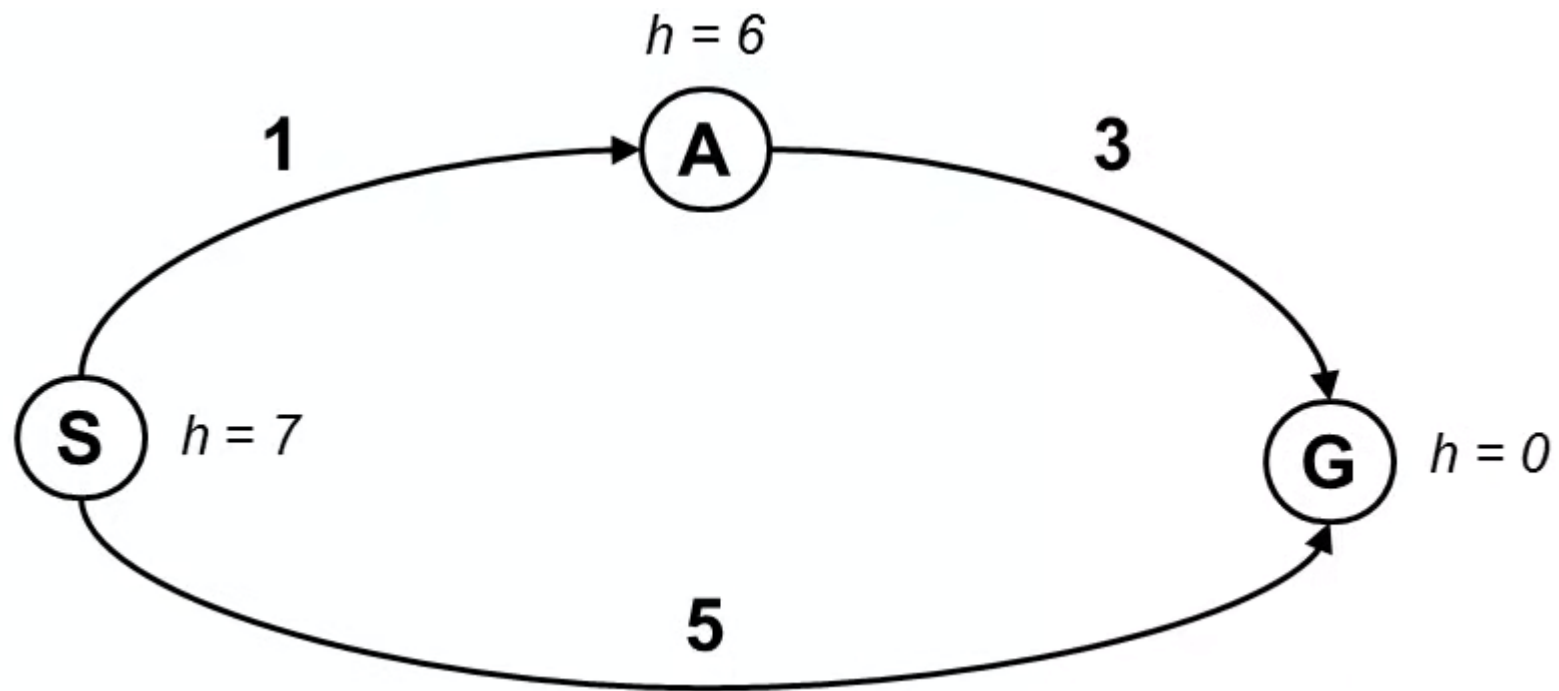
Otro ejemplo





¿Es A^* óptimo?

Otro ejemplo



Para la otra vez...

- Más allá de búsqueda clásica

The End.